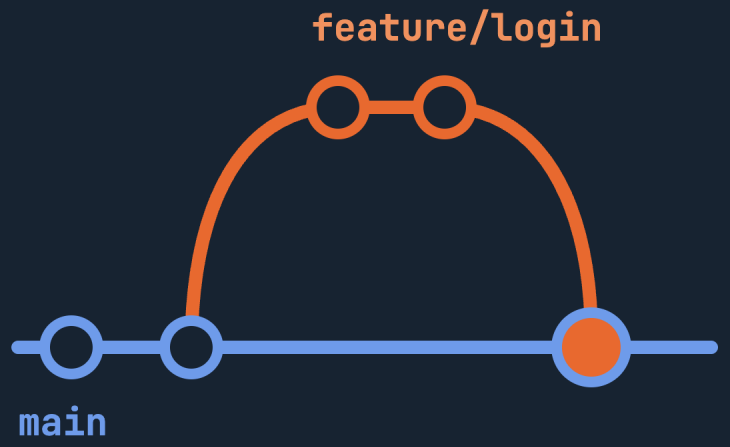
ZÁKLADY PRE KAŽDÉHO

Git a GitHub

Od klikania vo VS Code cez príkazy v termináli až po prácu s AI agentmi

Tomáš Kanko



PLÁN

O čom to dnes bude

01
Git a commit
Čo je Git a ako uložiť snímku

02
Vetvy a merge
Paralelná práca a jej spájanie

03
GitHub a PR
Push, pull a pull requesty

04
Stash a záchrana
Odloženie práce, vrátenie zmien

05
Zhrnutie
Celý postup a ťahák

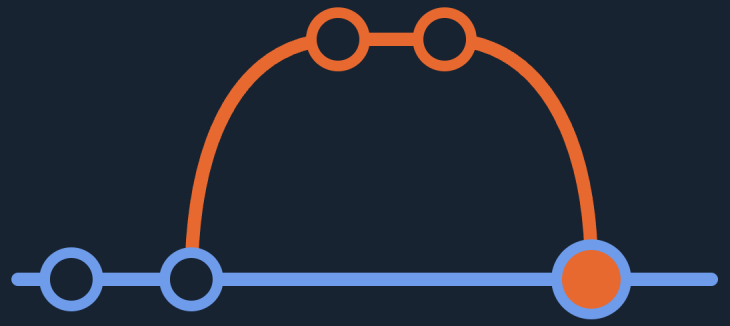
06
AI agenti
To isté, ale cez konverzáciu

Každý krok uvidíš dvakrát: VS CODE a TERMINÁL

01 / 06

Git a prvý commit

Čo Git vlastne robí a ako uložiť prvú verziu projektu



PROBLÉM

Poznáš toto?

DOKUMENTY / ŠKOLA / PROJEKT

- projekt.zip
- projekt_final.zip
- projekt_final_2.zip
- projekt_final_NAOZAJ.zip
- projekt_final_NAOZAJ_oprava.zip
- projekt_final_NAOZAJ_v3 (1).zip

Ktorá verzia je tá správna?

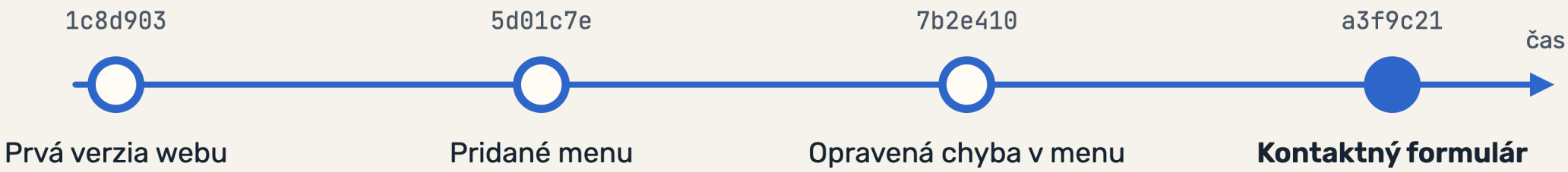
Čo sa medzi nimi zmenilo?

Kto mi prepísal môj kód?

Git rieši presne tento chaos.

ČO JE GIT

Git je fotoalbum tvojho projektu



Snímka = commit

Uloží stav všetkých súborov aj s popisom: kto, kedy a prečo.

Funguje lokálne

Celá história je na tvojom počítači, aj bez internetu.

Návrat kedykoľvek

Vieš sa pozrieť na ľubovoľnú starú verziu alebo ju obnoviť.

GIT ≠ GITHUB

Git je nástroj, GitHub je služba

Git

Program na tvojom počítači

- Sleduje zmeny v súboroch
- Funguje aj bez internetu
- Ovládaš ho príkazmi alebo v editore

GitHub

Webová služba na internete

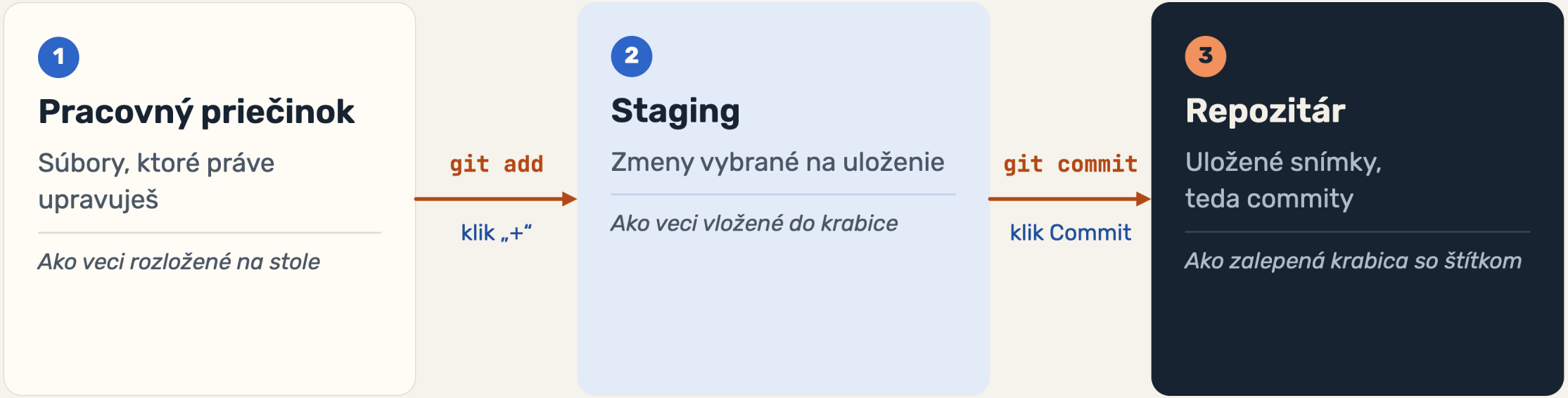
- Uchováva kópiu repozitára online
- Zdieľanie a spolupráca v tíme
- Pull requesty, review, issues

Podobné služby: GitLab, Bitbucket

Git je fotoaparát. GitHub je online album, kam fotky nahráš a ukážeš ich ostatným.

AKO TO FUNGUJE

Každá zmena prejde tromi miestami

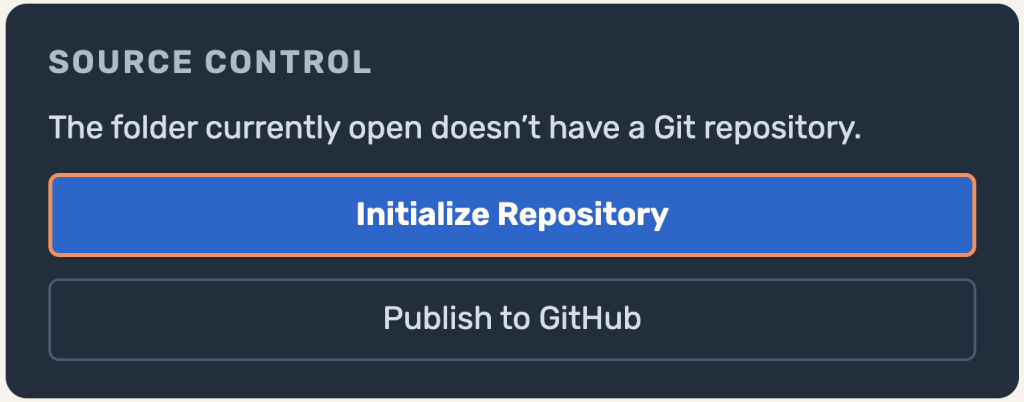


Staging ti dovolí uložiť len časť zmien, napríklad 2 súbory z 5, a zvyšok nechať na neskôr.

KROK 1 • REPOZITÁR

Začíname: založ repozitár

VS CODE



1. Otvor priečinok projektu: **File → Open Folder**
2. Otvor **Source Control** (Ctrl+Shift+G)
3. Klikni na **Initialize Repository**

TERMINÁL

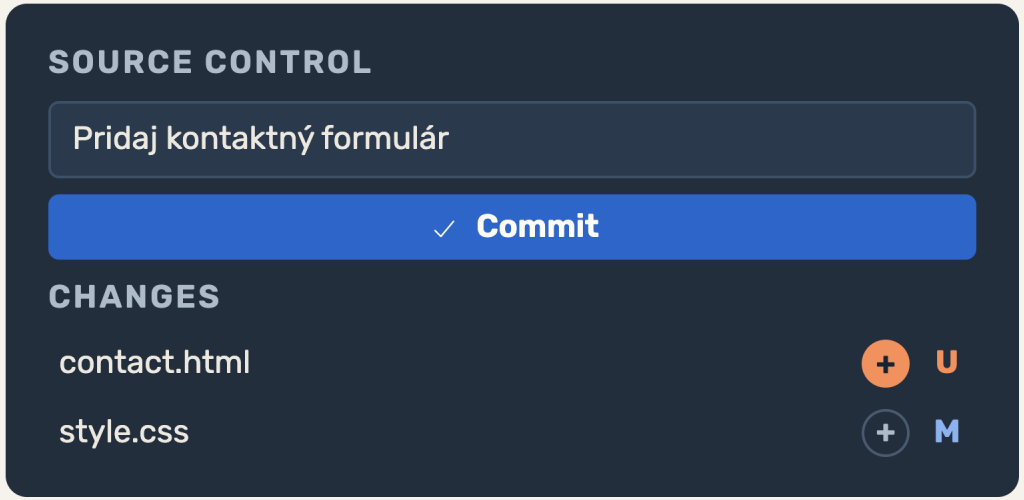
```
# raz na počítači: kto si
$ git config --global user.name "Ján Novák"
$ git config --global user.email jan@mail.sk
# v priečinku projektu
$ cd moj-web
$ git init
Initialized empty Git repository
```

git config nastavíš len raz. Meno a email sa zapíšu ku každému tvojmu commitu.

KROK 2 · COMMIT

Commit: ulož snímku projektu

VS CODE



1. Pri súbore klikni na **„+“** (Stage Changes)
2. Napíš krátku správu, čo si urobil
3. Klikni na ✓ **Commit**

TERMINÁL

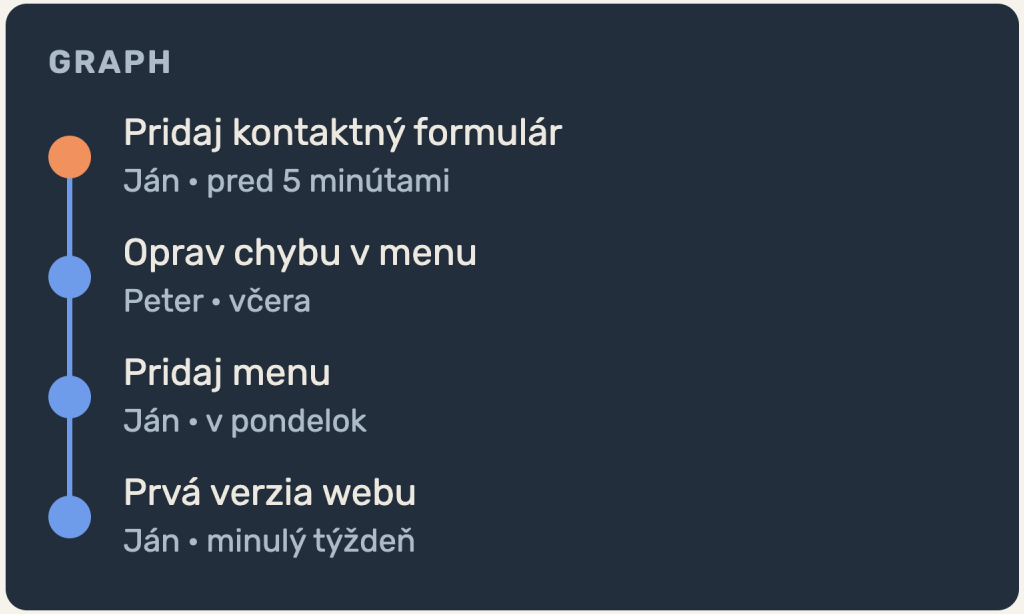
```
$ git status
# ukáže zmenené a nové súbory
$ git add contact.html style.css
# alebo všetko naraz: git add .
$ git commit -m "Pridaj kontaktný formulár"
[main a3f9c21] Pridaj kontaktný formulár
2 files changed, 48 insertions(+)
```

-m znamená message: krátka správa, ktorá ku commitu patrí.

KROK 3 • HISTÓRIA

Pozri sa do histórie

VS CODE



1. V Source Control rozbaľ sekciu **Graph**
2. Klik na commit ukáže, čo sa v ňom zmenilo

TERMINÁL

```
$ git log --oneline
a3f9c21 Pridaj kontaktný formulár
7b2e410 Oprav chybu v menu
5d01c7e Pridaj menu
1c8d903 Prvá verzia webu
$ git show a3f9c21
# ukáže presne, čo sa zmenilo
```

Každý commit má jedinečný kód (hash). Na odkazovanie stačí prvých 7 znakov.

TIP

Dobrá správa ušetrí čas celému tímu

TAKTO NIE	TAKTO ÁNO
`fix`	`Oprav pád formulára pri prázdnom emaile`
`zmeny`	`Pridaj validáciu kontaktného formulára`
`asdfgh`	`Zmeň farbu tlačidiel podľa nového dizajnu`
`piatok večer 2`	`Odstráň nepoužívané CSS štýly`

Píš, čo commit urobí

Začni slovesom: Pridaj…, Oprav…, Odstráň…, Zmeň…

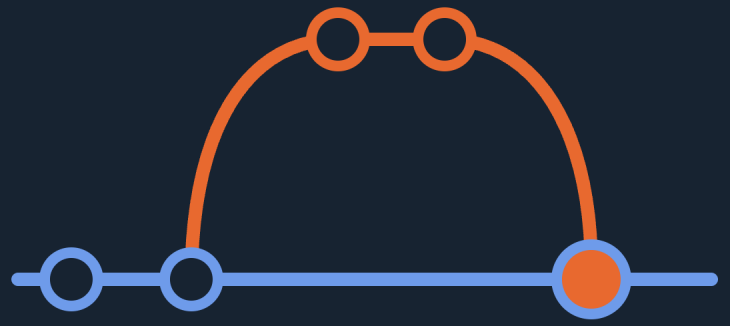
Jeden commit, jedna zmena

Malé commity sa ľahšie čítajú, kontrolujú aj vracajú.

02 / 06

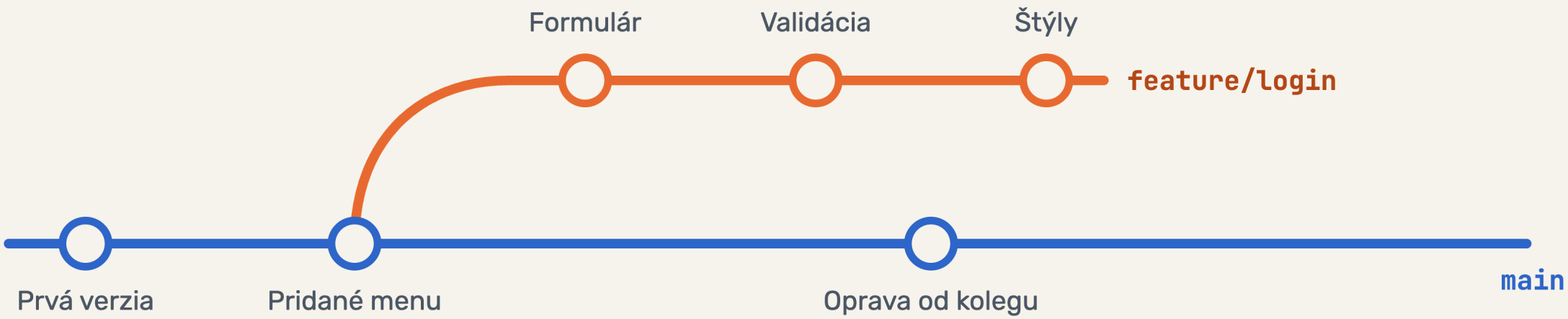
Vetvy a merge

Ako pracovať na viacerých veciach naraz a nič pritom nerozbiť



VETVY

Vetva: bezpečné miesto na experimenty



main

Hlavná, stabilná verzia. Tú vidia používatelia.

feature/login

Tvoja dielňa. Čo tu rozbiješ, main neovplyvní.

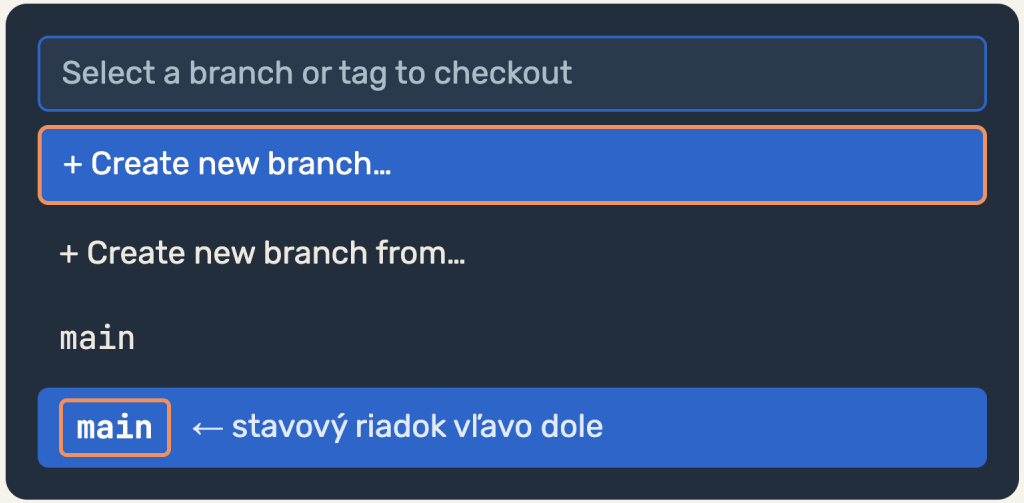
Viac vetiev naraz

Každá úloha má vlastnú vetvu: feature/…, fix/…

VETVY V PRAXI

Vytvor vetvu a prepínaj sa medzi nimi

VS CODE



1. Klikni na názov vetvy vľavo dole
2. Vyber **+ Create new branch...**
3. Zadaj názov, napríklad **feature/login**

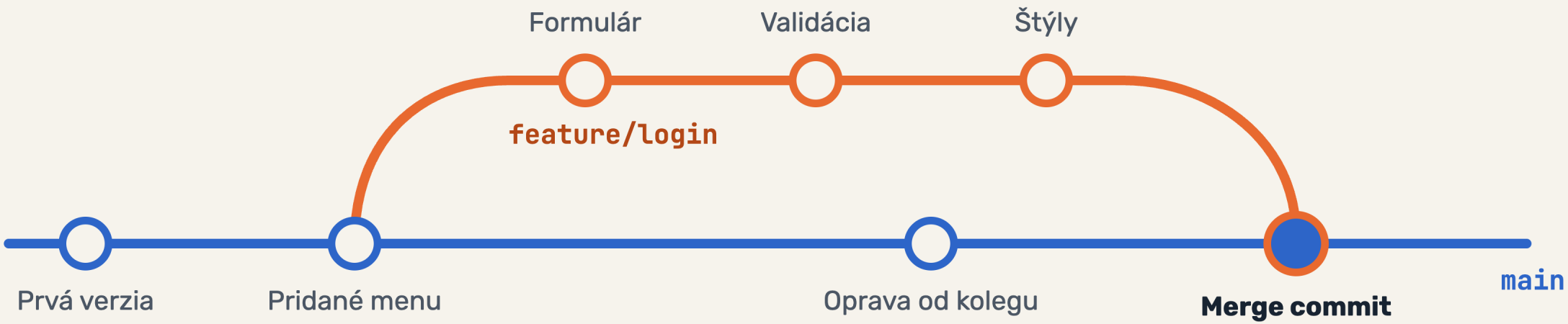
TERMINÁL

```
$ git branch
* main
$ git switch -c feature/login
Switched to a new branch 'feature/login'
$ git switch main
Switched to branch 'main'
# staršie: git checkout -b feature/login
```

-c znamená create: vytvor vetvu a rovno sa na ňu prepni. Vo VS Code sa prepneš klikom na názov vetvy.

MERGE

Merge spojí prácu z vetvy späť do main



Výsledok

main teraz obsahuje formulár aj opravu od kolegu.

Fast-forward

Ak sa main medzitým nezmenil, Git len posunie ukazovateľ.

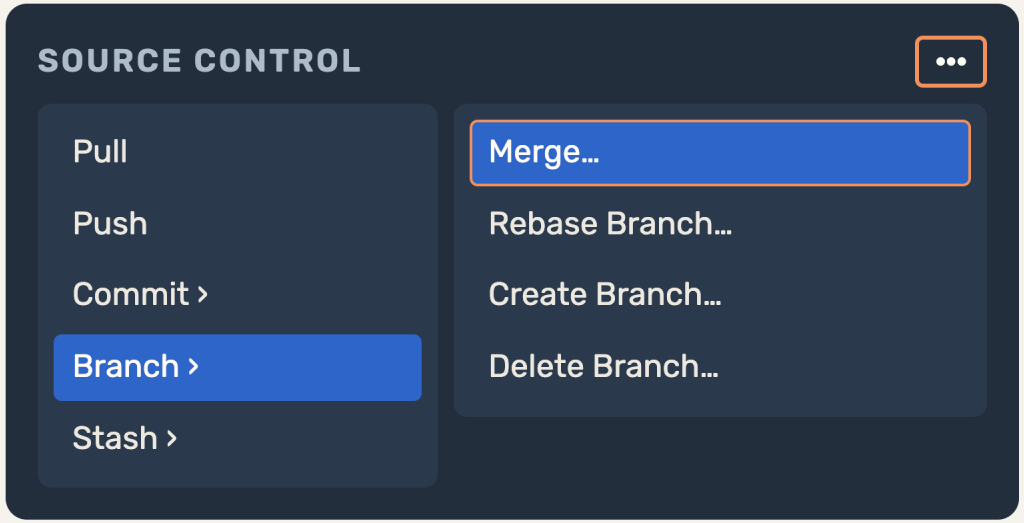
Po merge

Vetvu môžeš zmazať. Práca ostáva v main.

MERGE V PRAXI

Zmerguj vetvu do main

VS CODE



1. Prepni sa na **main** (klik na názov vetvy)
2. **„•••“ → Branch → Merge…**
3. Vyber vetvu **feature/login**

TERMINÁL

```
$ git switch main
$ git merge feature/login
Merge made by the 'ort' strategy.
# po merge vetvu uprac
$ git branch -d feature/login
Deleted branch feature/login
```

⚠ Merguješ vždy **do** vetvy, na ktorej práve stojíš.

KONFLIKT

Konflikt: dve zmeny v tom istom riadku

ČO UVIDÍŠ V SÚBORE index.html

Accept Current | Accept Incoming | Accept Both

```
<<<<<<< HEAD
<h1>Vitajte na našom webe</h1>
=======
<h1>Ahoj, vitaj u nás!</h1>
>>>>>>> feature/uvod
```

HEAD = tvoja vetva **feature/uvod = prichádza**

Konflikt nie je chyba. Git sa len pýta: ktorú verziu chceš?

VS CODE

1. Súbor nájdeš pod **Merge Changes**
2. Klikni **Accept Current**, **Incoming** alebo **Both**
3. Ulož súbor, klikni **„+“** a potom ✓ **Commit**

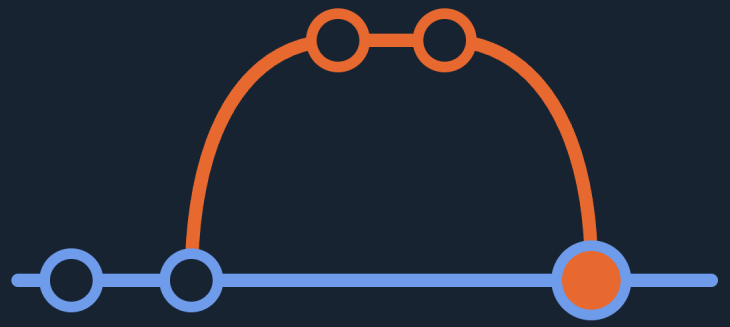
TERMINÁL

```
$ git merge feature/uvod
CONFLICT (content) in index.html
$ git add index.html
$ git commit
```

03 / 06

GitHub a pull requesty

Ako svoju prácu zdieľať s tímom: push, pull a pull requesty



REMOTE

Tvoj počítač, GitHub a kolegovia



Commit sa deje len u teba. Na GitHub sa dostane až po **push**. Novinky od kolegov získaš cez **pull**.

PREPOJENIE S GITHUBOM

Dva spôsoby, ako začať s GitHubom

A Projekt mám u seba

Chcem ho nahrať na GitHub.

VS CODE Source Control → **Publish to GitHub**

```
$ git remote add origin \
    https://github.com/jan/web.git
$ git push -u origin main
```

B Projekt už je na GitHube

Chcem si ho stiahnuť k sebe.

VS CODE Ctrl+Shift+P → **Git: Clone**

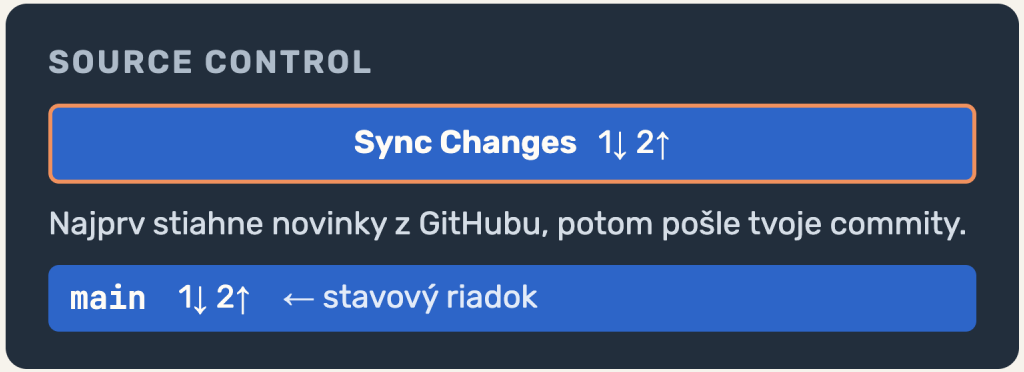
```
$ git clone \
    https://github.com/jan/web.git
$ cd web
```

-u origin main nastaví prepojenie raz. Potom už stačí písať len **git push**.

PUSH A PULL

Push posiela, pull sťahuje

VS CODE



1. Klikni na **Sync Changes**: pull aj push naraz
2. **1↓** čaká na stiahnutie, **2↑** na odoslanie
3. Novú vetvu pošleš cez **Publish Branch**

TERMINÁL

```
$ git pull
# stiahne a zapracuje (fetch + merge)
$ git push
# pošle tvoje commity na GitHub
$ git fetch
# len zistí, čo je nové; nič nemení
```

☆ Zlaté pravidlo: **pred prácou pull, po práci push.**

PULL REQUEST

Pull request = žiadosť o zlúčenie

01
Vetva
commity na feature/kontakt

→

02
Push
vetva je už aj na GitHube

→

03
Otvor PR
názov, popis, reviewer

→

04
Review
kolega komentuje a schváli

→

05
Merge
zmeny sa dostanú do main

Pridaj kontaktný formulár #42 Open

jan chce zlúčiť 3 commity z **feature/kontakt** do **main**

peter: Pridaj ešte kontrolu prázdneho mena.

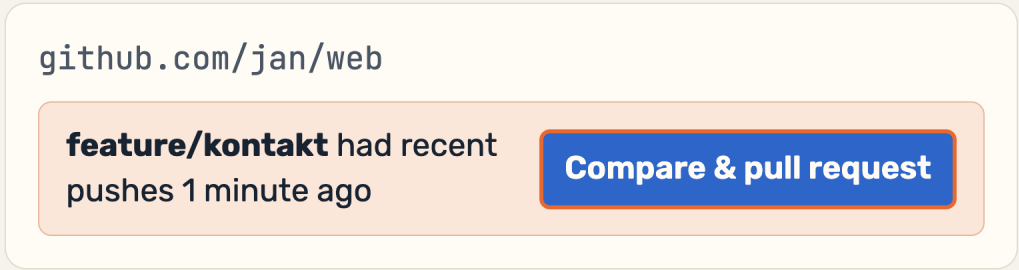
peter schválil zmeny

Merge pull request

PULL REQUEST V PRAXI

Ako otvoriť pull request

NA GITHUBE



1. Po push klikni na **Compare & pull request**
2. Napíš názov a popis, vyber reviewera
3. Klikni na **Create pull request**

VS CODE To isté zvládne rozšírenie **GitHub Pull Requests**.

TERMINÁL · GITHUB CLI

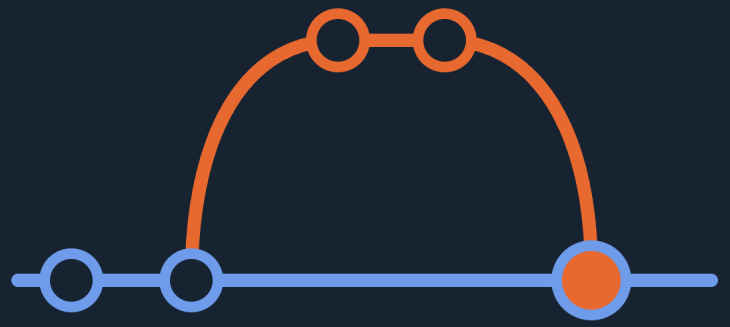
```
$ gh pr create --base main \
    --title "Kontaktný formulár" \
    --body "Pridáva formulár a validáciu"
https://github.com/jan/web/pull/42
$ gh pr list
$ gh pr merge 42 --merge
```

gh je oficiálny nástroj GitHubu pre terminál. Po merge si stiahni nový main: **git switch main** a **git pull**.

04 / 06

Stash a záchrana

Ako odložiť rozrobenú prácu a čo robiť, keď sa niečo pokazí



STASH

Stash: odlož rozrobenú prácu do šuplíka

Robíš na formulári a zrazu: „Na webe je chyba, oprav ju hneď!“

Polovičatú prácu commitnúť nechceš. Zahodiť ju tiež nie.

`git stash`

Odlož

Rozrobené zmeny idú do šuplíka.

→

`git switch main`

Oprav chybu

Oprava, commit, push.

→

`git switch -`

Vráť sa

Späť na svoju vetvu („-“ = predošlá).

→

`git stash pop`

Vyber

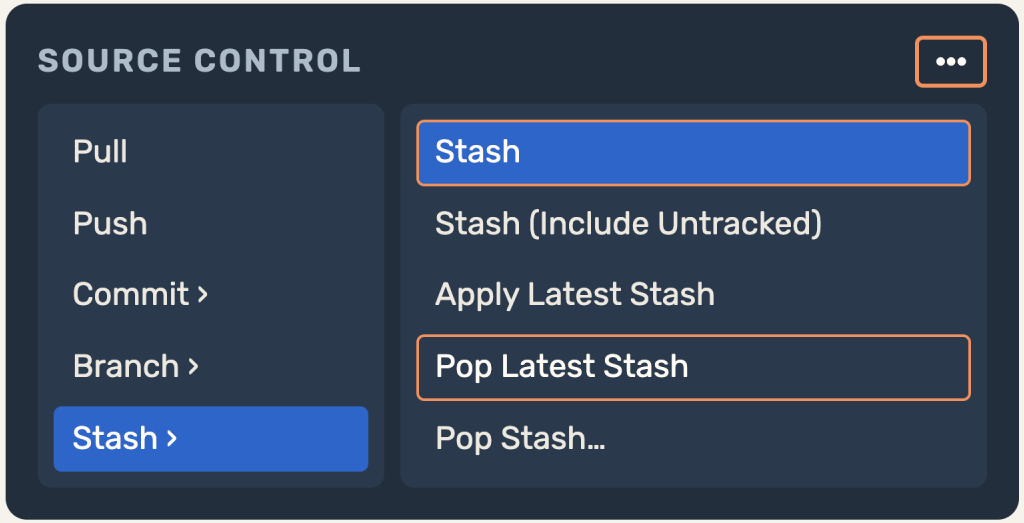
Zmeny sú späť, pokračuješ ďalej.

Do šuplíka môžeš odložiť aj viac vecí naraz a vyberať ich neskôr.

STASH V PRAXI

Ako na stash

VS CODE



1. „•••“ → **Stash** → **Stash**
2. Prepni vetvu, oprav chybu, commitni
3. Na svojej vetve: **Pop Latest Stash**

TERMINÁL

```
$ git stash
Saved working directory and index state
$ git switch main
# … oprava, commit, push …
$ git switch -
$ git stash pop
$ git stash list
# čo všetko je v šuplíku
```

Nové súbory, ktoré Git ešte nesleduje, pribaľ cez **git stash -u** (vo VS Code: Include Untracked).

ZÁCHRANNÉ BRZDY

Keď sa niečo pokazí

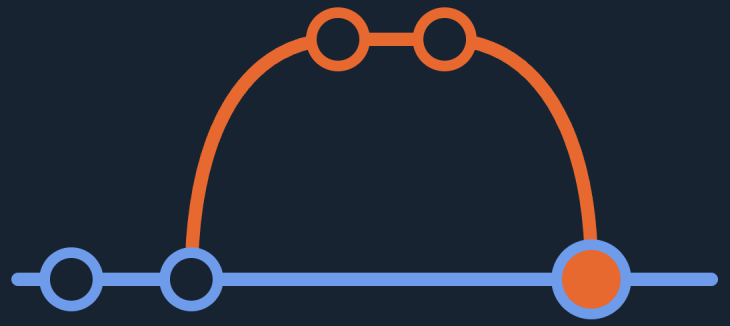
SITUÁCIA	VS CODE	PRÍKAZ
Zahodiť neuložené zmeny v súbore	Discard Changes (↶)	`git restore index.html`
Vrátiť súbor zo stagingu	Unstage Changes (–)	`git restore --staged index.html`
Doplniť niečo do posledného commitu	Commit ▾ → Commit (Amend)	`git commit --amend`
Vrátiť commit, ktorý už je na GitHube	najjednoduchšie príkazom	`git revert a3f9c21`
Niektoré súbory nemajú ísť do Gitu	súbor .gitignore	`echo ".env" >> .gitignore`

⚠ **Pozor:** restore a Discard zahodia zmeny natrvalo. **Revert** je bezpečný: pridá nový commit, ktorý zmenu vráti.

05 / 06

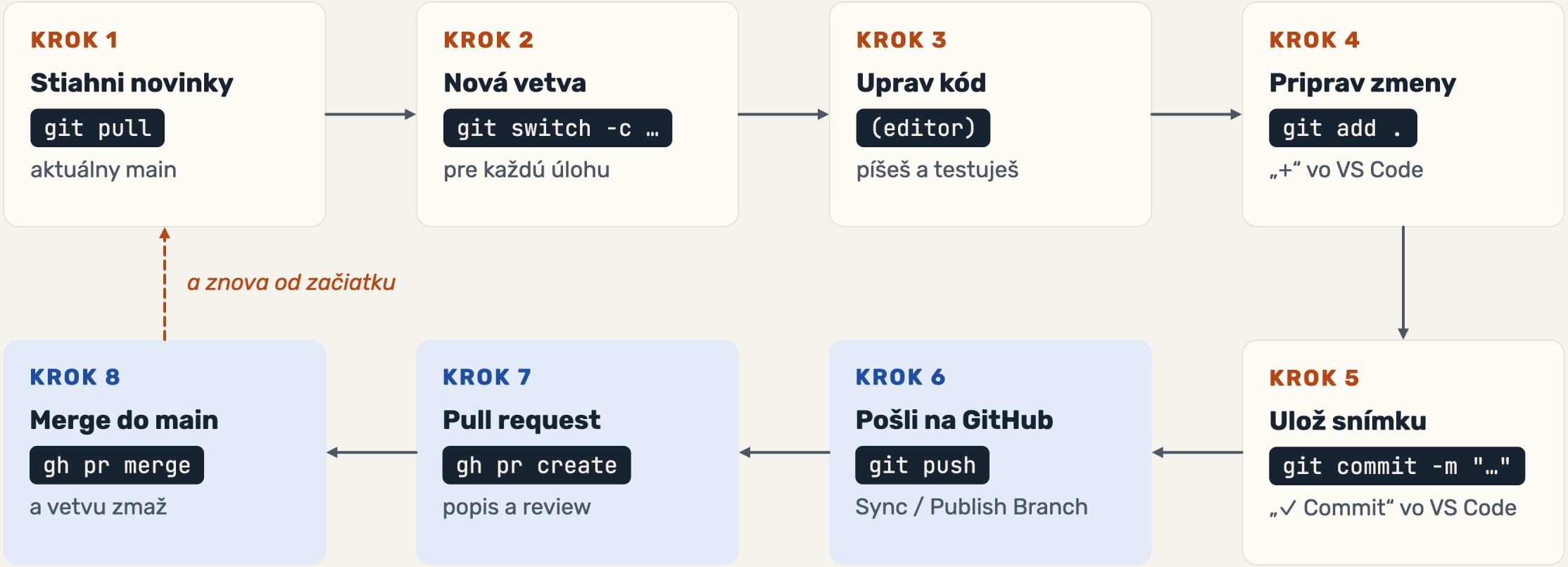
Zhrnutie

Celý postup na jednom obrázku a ťahák príkazov



ZHRNUTIE

Celý postup na jednom obrázku



ŤAHÁK

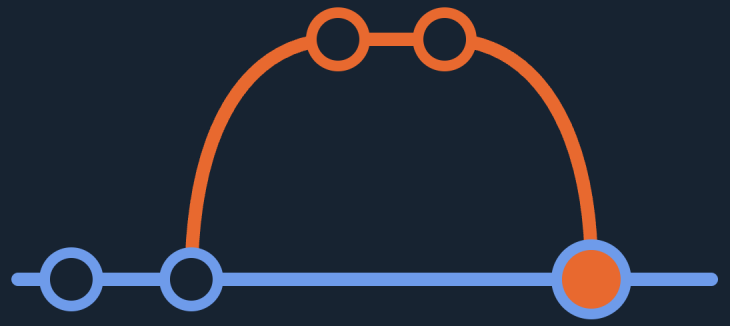
VS Code a príkazy vedľa seba

ČO CHCEM	VS CODE	TERMINÁL
Pozrieť zmeny	panel Source Control	`git status`
Pripraviť súbor	„+“ pri súbore	`git add súbor`
Uložiť snímku	✓ Commit	`git commit -m "…"`
Nová vetva	názov vetvy vľavo dole	`git switch -c vetva`
Zlúčiť vetvu	„•••“ → Branch → Merge…	`git merge vetva`
Poslať / stiahnuť	Sync Changes	`git push / git pull`
Odložiť prácu	„•••“ → Stash	`git stash / stash pop`
Pull request	GitHub web alebo rozšírenie	`gh pr create`

06 / 06

To isté cez AI agentov

Neklikáš, nepíšeš príkazy. Povieš, čo chceš, a skontroluješ výsledok.



NOVÝ SPÔSOB

Tri spôsoby, ten istý Git

VS Code

Klikáš na tlačidlá.

`„+“ → Commit → Sync`

→

Terminál

Píšeš príkazy.

```
git commit -m "…"
```

→

AI agent

Opíšeš, čo chceš. Agent spustí príkazy, ty ich skontroluješ.

`„Commitni to a otvor PR.“`

Agentom môže byť napríklad Claude Code, OpenAI Codex alebo GitHub Copilot v režime agenta. Pod kapotou robí presne to, čo sme sa práve naučili.

KONVERZÁCIA 1 / 4

Nová vetva a commit

„Vytvor vetvu feature/kontakt a do contact.php pridaj kontrolu, či je email zadaný správne. Potom to commitni.“

AGENT

Vytváram vetvu a upravujem formulár.

```
$ git switch -c feature/kontakt
# upravený contact.php (+14 -2)
$ git add contact.php
$ git commit -m "Pridaj validáciu emailu"
```

Hotovo: commit a3f9c21 je na feature/kontakt. Pushnúť na GitHub?

VŠIMNI SI

- Modrá bublina si ty, biela je agent.
- ✓ Zadanie je konkrétne: vetva, súbor aj cieľ.
- </> Agent spúšťa tie isté príkazy, ktoré už poznáš.
- Pred push sa opýta. To je krok, ktorý ide von z tvojho PC.

KONVERZÁCIA 2 / 4

Push a pull request

„Áno, pushni to a otvor pull request do main. Do popisu napíš, čo sa zmenilo a ako to otestovať.“

AGENT

Posielam vetvu na GitHub a otváram pull request.

```
$ git push -u origin feature/kontakt
$ gh pr create --base main \
    --title "Validácia emailu vo formulári" \
    --body "Čo sa zmenilo: … Ako testovať: …"
```

Pull request **#42** je otvorený a čaká na review.

VŠIMNI SI

- Stačí „Áno“. Agent vie, o čom sa bavíte.
- Popis PR napíše agent, ty ho len skontroluješ.
- Review stále robí človek z tímu.

KONVERZÁCIA 3 / 4

Rýchla oprava a stash

„Na main je chyba: v pätičke sa zobrazuje zlý rok. Oprav to, ale nestrať moju rozrobenú prácu.“

AGENT

Odložím tvoje zmeny a chybu opravím na novej vetve.

```
$ git stash
$ git switch main && git pull
$ git switch -c fix/rok-v-paticke
# upravený footer.php (+1 -1)
$ git commit -am "Oprav rok v pätičke"
$ git switch feature/kontakt && git stash pop
```

Oprava je na fix/rok-v-paticke a tvoja práca je späť. Otvoriť aj PR?

VŠIMNI SI

- Ty si povedal čo. Agent vymyslel ako.
- ✓ Oprava má vlastnú vetvu, nemieša sa s formulárom.
- Rozrobená práca sa nestratila: stash a stash pop.

KONVERZÁCIA 4 / 4

Konflikt pri merge

„Zmerguj main do mojej vetvy. Ak bude konflikt, najprv sa ma opýtaj.“

AGENT

```
$ git merge main
CONFLICT (content) in config.php
```

Konflikt v config.php: na main sa zmenila adresa API, u teba timeout. Navrhujem ponechať obe zmeny. Súhlasíš?

„Áno, ponechaj obe.“

Agent: konflikt vyriešený, merge dokončený (git add + git commit)

VŠIMNI SI

- Pravidlo „najprv sa opýtaj“ dáš priamo do zadania.
- Agent vysvetlí, čo sa zmenilo na ktorej strane.
- Rozhodnutie ostáva na tebe.

AKO SA S AGENTOM ROZPRÁVAŤ

Štyri pravidlá dobrej spolupráce

Buď konkrétny

Povedz vetvu, súbor aj cieľ.

`„Pridaj validáciu emailu do contact.php“`

~~`„sprav to nejako lepšie“`~~

Stanov hranice

„Nepushuj bez opýtania.“ „Nikdy nerob force push na main.“

Trvalé pravidlá zapíš do súboru **AGENTS.md** alebo **CLAUDE.md** v projekte.

Kontroluj výsledok

Pred schválením si pozri **git diff** a pull request. Agent sa môže mýliť.

Malé kroky

Jedna úloha = jedna vetva = jeden pull request. Menšie zmeny sa ľahšie kontrolujú.

KEDY ČO POUŽIŤ

Neexistuje jediný správny spôsob

VS Code

Keď chceš vidieť zmeny, riešiť konflikty a ešte sa učíš.

Terminál

Keď chceš rýchlosť a presnosť, na serveri alebo v skriptoch.

AI agent

Keď má úloha viac krokov: vetva, commity, push, PR. Vždy s tvojou kontrolou.

Agent píše príkazy za teba.

Pochopenie a kontrola ostávajú na tebe.

DOMÁCA ÚLOHA

Vyskúšaj si to

Prejdi celý postup sám a potom ho zadaj AI agentovi. Porovnaj, čo spravil.

1 • repozitár | 2 • tri commity | 3 • vetva | 4 • merge | 5 • pull request | 6 • to isté s agentom

Ďakujem! Otázky?